Assignment 4

Arithmetic Function Library

Name: Arnav Singhal
MIS: 612203012
Final Year B. Tech. Comp. Div 1
Batch: B3

1. Introduction

Modern public-key cryptography, primality testing, and block-cipher design all rest on a small set of recurring number-theoretic and algebraic building blocks: fast modular exponentiation, the Euclidean and extended Euclidean algorithms, primitive roots of the multiplicative group modulo n, probabilistic primality testing, and arithmetic over finite fields. This assignment implements a single library that collects six such functions implementing the same.

The library is wrapped in an interactive, menu-driven program: it runs in an infinite loop that repeatedly displays a menu of the six operations plus an exit option, reads every value needed for the selected operation directly from the user, and returns to the menu after each operation so multiple computations can be performed in a single run without restarting the program.

2. Modular Exponentiation

Modular exponentiation computes a^b mod n efficiently without ever forming the (potentially enormous) value a^b directly. The library uses the square-and-multiply method: the exponent is processed one bit at a time, the base is repeatedly squared modulo n, and the result accumulates a factor of the current base whenever the corresponding bit of the exponent is 1. This reduces the cost from O(b) multiplications to O(log b).

Example: 7^560 mod 561 is computed in 10 squarings instead of 560 multiplications, since 560 requires 10 bits to represent.

3. Euclidean Algorithm for GCD

The Euclidean algorithm computes gcd(a, b) by repeatedly replacing the pair (a, b) with (b, a mod b) until the remainder reaches 0; the last nonzero remainder is the greatest common divisor. The table below traces the algorithm on gcd(252, 198):

Step	a	b	q = a div b	r = a mod b
1	252	198	1	54
2	198	54	3	36
3	54	36	1	18
4	36	18	2	0

Since the remainder reaches 0 when b = 18, gcd(252, 198) = 18.

4. Extended Euclidean Algorithm and Modular Inverse

The extended Euclidean algorithm augments the ordinary algorithm to also return integers x and y satisfying Bezout's identity a*x + b*y = gcd(a, b). When gcd(a, m) = 1, the coefficient x, reduced modulo m, is exactly the modular inverse of a modulo m, that is, the value satisfying a*x = 1 (mod m). The library's mod_inverse() function wraps extended_gcd() for this purpose and returns None when no inverse exists.

Example: computing the inverse of 17 modulo 3120 (the values used to derive an RSA private exponent when e = 17 and phi(n) = 3120) unwinds as follows:

Call	a	b
extended_gcd(17, 3120)	17	3120
extended_gcd(3120, 17)	3120	17
extended_gcd(17, 9)	17	9
extended_gcd(9, 8)	9	8
extended_gcd(8, 1)	8	1
extended_gcd(1, 0)	1	0

Unwinding the recursion back up gives x = -367, which reduces to 2753 modulo 3120. Checking: (17 * 2753) mod 3120 = 1, confirming 2753 is the modular inverse of 17 modulo 3120.

5. Primitive Roots

An integer g is a primitive root modulo n if every integer coprime to n is congruent to some power of g modulo n, equivalently if the multiplicative order of g modulo n equals phi(n). Primitive roots exist only for n equal to 1, 2, 4, p^k, or 2p^k for an odd prime p.

The library tests each candidate g in the range [2, n) with gcd(g, n) = 1 by checking that g^(phi(n)/q) is not congruent to 1 modulo n for every prime factor q of phi(n); if this holds for all such q, g is a primitive root.

Example: for n = 23, phi(23) = 22 = 2 * 11, so each candidate is checked against the exponents 22/2 = 11 and 22/11 = 2. The resulting primitive roots are 5, 7, 10, 11, 14, 15, 17, 19, 20, and 21, a total of phi(22) = 10 roots, matching the theoretical count of phi(phi(n)) primitive roots modulo a prime n.

6. Miller-Rabin Primality Test

The Miller-Rabin test is a probabilistic primality test based on writing n - 1 as 2^r * d with d odd, then checking whether a random witness a satisfies either a^d = 1 (mod n) or a^(2^j * d) = n - 1 (mod n) for some 0 <= j < r. If neither holds, n is definitely composite; if one holds, n is probably prime for that witness. Repeating the test with k independently chosen witnesses drives the probability of a false positive down to at most 4^-k.

The library runs k = 20 rounds by default, which is more than sufficient for practical use, including against Carmichael numbers such as 561 and 341 that pass Fermat's test for every base coprime to them but are correctly rejected by Miller-Rabin.

7. Polynomial Arithmetic on GF(2^8)

GF(2^8) is the finite field used by AES, where each byte is viewed as a degree-7 polynomial with coefficients in GF(2) (that is, each bit is a coefficient of 0 or 1). Addition in this field is bitwise XOR, since coefficients are added modulo 2 with no carry. Multiplication is carryless polynomial multiplication followed by reduction modulo the irreducible polynomial x^8 + x^4 + x^3 + x + 1 (represented as the byte pattern 0x11B), which keeps every result within 8 bits.

The library's gf_mult() function implements this with a shift-and-add loop: for each of the 8 bits of the second operand, if the bit is 1 the current (shifted) first operand is XORed into the result; the first operand is then left-shifted by one bit, and if that shift overflows past bit 7, the irreducible polynomial's byte pattern is XORed in to reduce it back into the field. Repeated multiplication (gf_pow) combined with Fermat's little theorem for fields, a^254 = a^-1 for nonzero a in GF(2^8), gives the multiplicative inverse used in gf_inverse().

Example: multiplying a = 0x57 (x^6 + x^4 + x^2 + x + 1) by b = 0x83 (x^7 + x + 1), the standard worked example from the AES specification, proceeds bit by bit through b as follows:

i	x before shift	bit i of b	overflow	x after reduction	running result
0	0x57	1	no	0xAE	0x57
1	0xAE	1	yes	0x47	0xF9
2	0x47	0	no	0x8E	0xF9
3	0x8E	0	yes	0x07	0xF9
4	0x07	0	no	0x0E	0xF9
5	0x0E	0	no	0x1C	0xF9
6	0x1C	0	no	0x38	0xF9
7	0x38	1	no	0x70	0xC1

The final running result after all 8 bits is 0xC1, so 0x57 * 0x83 = 0xC1 in GF(2^8), matching the well-known AES test vector for this multiplication.

8. Program Implementation

8.1 Library Design

All six capabilities are implemented as standalone functions in arithmetic_lib.py so each can be imported and reused independently: mod_exp() for modular exponentiation, gcd() for the Euclidean algorithm, extended_gcd() and mod_inverse() for the extended algorithm and modular inverses, primitive_roots() (supported by euler_phi(), prime_factors(), and has_primitive_root()) for finding primitive roots, miller_rabin() for primality testing, and gf_add(), gf_mult(), gf_pow(), and gf_inverse() for GF(2^8) polynomial arithmetic under the AES irreducible polynomial.

8.2 Menu-Driven Design and User Input

The main() function runs an infinite while loop: on every iteration it prints the menu, via print_menu(), and reads a choice with read_int(), a helper that keeps re-prompting until a valid integer is entered. Based on the choice, control is dispatched to one of six menu functions, one per library operation, each of which prompts for exactly the values that operation needs (for example, menu_mod_exp() asks for a, b and n, while menu_gf() first asks the user to pick addition, multiplication or inverse and then reads one or two field elements accordingly). The GF(2^8) elements are read with read_gf_byte(), which accepts either decimal or 0x-prefixed hexadecimal input and re-prompts if the value does not fit in a single byte (0 to 255). Selecting option 7 prints a closing message and breaks out of the loop, ending the program; any other invalid choice is reported and the menu is simply shown again.

8.3 Code and Output

```
import random


def mod_exp(base, exp, mod):
    if mod == 1:
        return 0
    result = 1
    base = base % mod
    while exp > 0:
        if exp & 1:
            result = (result * base) % mod
        exp >>= 1
        base = (base * base) % mod
    return result


def gcd(a, b):
    a, b = abs(a), abs(b)
    while b != 0:
        a, b = b, a % b
    return a


def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x1, y1 = extended_gcd(b, a % b)
    x = y1
    y = x1 - (a // b) * y1
```

```
    return g, x, y


def mod_inverse(a, m):
    g, x, _ = extended_gcd(a, m)
    if g != 1:
        return None
    return x % m


def euler_phi(n):
    result = n
    p = 2
    temp = n
    while p * p <= temp:
        if temp % p == 0:
            while temp % p == 0:
                temp //= p
            result -= result // p
        p += 1
    if temp > 1:
        result -= result // temp
    return result


def prime_factors(n):
    factors = []
    temp = n
    p = 2
    while p * p <= temp:
        if temp % p == 0:
            factors.append(p)
            while temp % p == 0:
                temp //= p
        p += 1
    if temp > 1:
        factors.append(temp)
    return factors


def has_primitive_root(n):
```

```
    if n in (1, 2, 4):
        return True
    if n % 2 == 0:
        n //= 2
    if n % 2 == 0:
        return False
    factors = prime_factors(n)
    return len(factors) == 1


def primitive_roots(n):
    if not has_primitive_root(n):
        return []
    phi = euler_phi(n)
    phi_factors = prime_factors(phi)
    roots = []
    for g in range(2, n):
        if gcd(g, n) != 1:
            continue
        is_root = True
        for q in phi_factors:
            if mod_exp(g, phi // q, n) == 1:
                is_root = False
                break
        if is_root:
            roots.append(g)
    return roots


def miller_rabin(n, k=20):
    if n in (2, 3):
        return True
    if n < 2 or n % 2 == 0:
        return False

    r, d = 0, n - 1
    while d % 2 == 0:
        r += 1
        d //= 2

    for _ in range(k):
```

```
            a = random.randrange(2, n - 1)
            x = mod_exp(a, d, n)
            if x == 1 or x == n - 1:
                continue
            composite = True
            for _ in range(r - 1):
                x = (x * x) % n
                if x == n - 1:
                    composite = False
                    break
            if composite:
                return False
        return True


GF_MOD = 0b100011011


def gf_add(a, b):
    return a ^ b


def gf_mult(a, b):
    result = 0
    x = a
    y = b
    for _ in range(8):
        if y & 1:
            result ^= x
        carry = x & 0x80
        x <<= 1
        if carry:
            x ^= GF_MOD
        x &= 0xFF
        y >>= 1
    return result & 0xFF


def gf_pow(a, exp):
    result = 1
    base = a
```

```
    while exp > 0:
        if exp & 1:
            result = gf_mult(result, base)
        base = gf_mult(base, base)
        exp >>= 1
    return result


def gf_inverse(a):
    if a == 0:
        return 0
    return gf_pow(a, 254)


def poly_to_str(a):
    if a == 0:
        return "0"
    terms = []
    for i in range(7, -1, -1):
        if a & (1 << i):
            if i == 0:
                terms.append("1")
            elif i == 1:
                terms.append("x")
            else:
                terms.append(f"x^{i}")
    return " + ".join(terms)


def read_int(prompt):
    while True:
        raw = input(prompt).strip()
        try:
            return int(raw)
        except ValueError:
            print("Please enter a valid integer.")


def read_gf_byte(prompt):
    while True:
        raw = input(prompt).strip()
```

```
        try:
            if raw.lower().startswith("0x"):
                value = int(raw, 16)
            else:
                value = int(raw)
        except ValueError:
            print("Please enter a valid byte, as decimal or hex (e.g. 0x57).")
            continue
        if 0 <= value <= 255:
            return value
        print("Value must be between 0 and 255 (a single byte).")


def menu_mod_exp():
    base = read_int("Enter base a: ")
    exp = read_int("Enter exponent b: ")
    mod = read_int("Enter modulus n: ")
    print(f"{base}^{exp} mod {mod} = {mod_exp(base, exp, mod)}")


def menu_gcd():
    a = read_int("Enter first number a: ")
    b = read_int("Enter second number b: ")
    print(f"gcd({a}, {b}) = {gcd(a, b)}")


def menu_mod_inverse():
    a = read_int("Enter number a: ")
    m = read_int("Enter modulus m: ")
    inv = mod_inverse(a, m)
    if inv is None:
        print(f"No modular inverse exists for {a} mod {m} since gcd({a}, {m}) != 1")
    else:
        print(f"Inverse of {a} mod {m} = {inv}")
        print(f"Check: ({a} * {inv}) mod {m} = {(a * inv) % m}")


def menu_primitive_roots():
    n = read_int("Enter n: ")
    if n < 1:
        print("n must be a positive integer.")
```

```
            return
        roots = primitive_roots(n)
        if not roots:
            print(f"{n} has no primitive roots.")
        else:
            print(f"Primitive roots of {n}: {roots}")


def menu_miller_rabin():
        n = read_int("Enter the number to test: ")
        k = read_int("Enter the number of rounds (e.g. 20): ")
        if k < 1:
            print("Number of rounds must be at least 1.")
            return
        result = miller_rabin(n, k)
        print(f"{n} is prime: {result}")


def menu_gf():
        print("1. Addition (XOR)")
        print("2. Multiplication")
        print("3. Multiplicative Inverse")
        choice = read_int("Choose an operation: ")
        if choice == 1:
            a = read_gf_byte("Enter byte a (decimal or hex, e.g. 0x57): ")
            b = read_gf_byte("Enter byte b (decimal or hex, e.g. 0x83): ")
            result = gf_add(a, b)
            print(f"a = 0x{a:02X} ({poly_to_str(a)})")
            print(f"b = 0x{b:02X} ({poly_to_str(b)})")
            print(f"a + b = 0x{result:02X} ({poly_to_str(result)})")
        elif choice == 2:
            a = read_gf_byte("Enter byte a (decimal or hex, e.g. 0x57): ")
            b = read_gf_byte("Enter byte b (decimal or hex, e.g. 0x83): ")
            result = gf_mult(a, b)
            print(f"a = 0x{a:02X} ({poly_to_str(a)})")
            print(f"b = 0x{b:02X} ({poly_to_str(b)})")
            print(f"a * b = 0x{result:02X} ({poly_to_str(result)})")
        elif choice == 3:
            a = read_gf_byte("Enter byte a (decimal or hex, e.g. 0x57): ")
            inv = gf_inverse(a)
            print(f"a = 0x{a:02X} ({poly_to_str(a)})")
```

```
            print(f"inverse of a = 0x{inv:02X} ({poly_to_str(inv)})")
            print(f"check: (a * inverse of a) = 0x{gf_mult(a, inv):02X}")
        else:
            print("Invalid choice.")


def print_menu():
    print()
    print("Arithmetic Function Library")
    print("1. Modular Exponentiation (a^b mod n)")
    print("2. Euclidean Algorithm for GCD")
    print("3. Extended Euclidean Algorithm for Modular Inverse")
    print("4. Primitive Roots of a Number")
    print("5. Miller-Rabin Primality Test")
    print("6. Polynomial Arithmetic on GF(2^8)")
    print("7. Exit")


def main():
    while True:
        print_menu()
        choice = read_int("Enter your choice: ")
        print()
        if choice == 1:
            menu_mod_exp()
        elif choice == 2:
            menu_gcd()
        elif choice == 3:
            menu_mod_inverse()
        elif choice == 4:
            menu_primitive_roots()
        elif choice == 5:
            menu_miller_rabin()
        elif choice == 6:
            menu_gf()
        elif choice == 7:
            print("Exiting.")
            break
        else:
            print("Invalid choice, please select an option from the menu.")
```

```
if __name__ == "__main__":
    main()
```

Output:

```
arnav@arnav-HP-Laptop-15s-fq5xxx:~/Desktop/COEP/CNS/Assignment4$ python3 arithmetic_lib.py

Arithmetic Function Library
1. Modular Exponentiation (a^b mod n)
2. Euclidean Algorithm for GCD
3. Extended Euclidean Algorithm for Modular Inverse
4. Primitive Roots of a Number
5. Miller-Rabin Primality Test
6. Polynomial Arithmetic on GF(2^8)
7. Exit
Enter your choice: 1

Enter base a: 7
Enter exponent b: 560
Enter modulus n: 561
7^560 mod 561 = 1

Arithmetic Function Library
1. Modular Exponentiation (a^b mod n)
2. Euclidean Algorithm for GCD
3. Extended Euclidean Algorithm for Modular Inverse
4. Primitive Roots of a Number
5. Miller-Rabin Primality Test
6. Polynomial Arithmetic on GF(2^8)
7. Exit
Enter your choice: 2

Enter first number a: 252
Enter second number b: 198
gcd(252, 198) = 18
```

```
Arithmetic Function Library
1. Modular Exponentiation (a^b mod n)
2. Euclidean Algorithm for GCD
3. Extended Euclidean Algorithm for Modular Inverse
4. Primitive Roots of a Number
5. Miller-Rabin Primality Test
6. Polynomial Arithmetic on GF(2^8)
7. Exit
Enter your choice: 3

Enter number a: 17
Enter modulus m: 3120
Inverse of 17 mod 3120 = 2753
Check: (17 * 2753) mod 3120 = 1

Arithmetic Function Library
1. Modular Exponentiation (a^b mod n)
2. Euclidean Algorithm for GCD
3. Extended Euclidean Algorithm for Modular Inverse
4. Primitive Roots of a Number
5. Miller-Rabin Primality Test
6. Polynomial Arithmetic on GF(2^8)
7. Exit
Enter your choice: 4

Enter n: 23
Primitive roots of 23: [5, 7, 10, 11, 14, 15, 17, 19, 20, 21]
```

```
Arithmetic Function Library
1. Modular Exponentiation (a^b mod n)
2. Euclidean Algorithm for GCD
3. Extended Euclidean Algorithm for Modular Inverse
4. Primitive Roots of a Number
5. Miller-Rabin Primality Test
6. Polynomial Arithmetic on GF(2^8)
7. Exit
Enter your choice: 5

Enter the number to test: 561
Enter the number of rounds (e.g. 20): 20
561 is prime: False

Arithmetic Function Library
1. Modular Exponentiation (a^b mod n)
2. Euclidean Algorithm for GCD
3. Extended Euclidean Algorithm for Modular Inverse
4. Primitive Roots of a Number
5. Miller-Rabin Primality Test
6. Polynomial Arithmetic on GF(2^8)
7. Exit
Enter your choice: 6

1. Addition (XOR)
2. Multiplication
3. Multiplicative Inverse
Choose an operation: 2
Enter byte a (decimal or hex, e.g. 0x57): 0x57
Enter byte b (decimal or hex, e.g. 0x83): 0x83
a = 0x57 (x^6 + x^4 + x^2 + x + 1)
b = 0x83 (x^7 + x + 1)
a * b = 0xC1 (x^7 + x^6 + 1)
```

```
Arithmetic Function Library
1. Modular Exponentiation (a^b mod n)
2. Euclidean Algorithm for GCD
3. Extended Euclidean Algorithm for Modular Inverse
4. Primitive Roots of a Number
5. Miller-Rabin Primality Test
6. Polynomial Arithmetic on GF(2^8)
7. Exit
Enter your choice: 7

Exiting.
arnav@arnav-HP-Laptop-15s-fq5xxx:~/Desktop/COEP/CNS/Assignment4$
```

8.4 Demonstration Result

Running the program and exercising all six menu options against known values confirms correctness throughout: the modular exponentiation and GCD results match hand calculation; the modular inverse check (17 * 2753) mod 3120 = 1 confirms correctness; the ten primitive roots of 23 match the expected count of phi(phi(23)) = 10; Miller-Rabin correctly reports 561 as composite; and the GF(2^8) multiplication of 0x57 and 0x83 reproduces the AES specification's worked example result of 0xC1. Choosing option 7 at the menu cleanly exits the infinite loop.

9. Conclusion

This assignment implemented a reusable arithmetic library covering modular exponentiation, the Euclidean and extended Euclidean algorithms, primitive root computation, Miller-Rabin primality testing, and GF(2^8) polynomial arithmetic under the AES irreducible polynomial, wrapped in a menu-driven program that loops indefinitely and takes all operands from the user until the exit option is chosen. Each function was validated, confirming that the implementations are correct.